

Module – Arrays (Declaration, Initialization, Multi-Dimensional)

What Are Arrays?

Key Points

- Arrays store multiple values of the same type in a single variable.
- Elements are indexed starting from 0.
- Arrays have a fixed size once created.
- Can hold primitive or reference types.

Code Snippet

```
int[] numbers;           // declaration
numbers = new int[5];    // allocation (default values = 0)
```

Exercise 01 – ArrayDeclaration.java

Declare an integer array of size 4. Print all elements using a for loop.

Array Declaration Styles

Key Points

- Declaration and creation can be combined or separate.
- Two valid syntaxes for declaration:
 - `int[] arr;` (preferred style)
 - `int arr[];` (legal but less common)

Code Snippet

```
String[] names = new String[3];  
double prices[];  
prices = new double[5];
```

Exercise 02 – ArrayStyles.java

Create a `String[]` to store 3 city names. Assign values and print them.

Initialization with Values

Key Points

- Arrays can be initialized at creation time using curly braces {}.
- Array length is inferred automatically.
- Once initialized, size cannot change.

Code Snippet

```
char[] vowels = {'A', 'E', 'I', 'O', 'U'};  
int[] scores = {90, 85, 88, 92};  
System.out.println("First vowel: " + vowels[0]);  
System.out.println("Array length: " + scores.length);
```

Exercise 03 – ArrayInitialization.java

Create an `int[]` with five exam marks. Calculate and print their average.

Accessing and Iterating

Key Points

- Access elements via index: `array[index]`.
- Common loops: `for` and `for-each`.
- Accessing out-of-range index throws `ArrayIndexOutOfBoundsException`.

Code Snippet

```
int[] nums = {10, 20, 30};
for (int i = 0; i < nums.length; i++) {
    System.out.println(nums[i]);
}
for (int n : nums) {
    System.out.println("Value: " + n);
}
```

Exercise 04 – ArrayIteration.java

Create a `double[]` with 4 numbers. Print the sum using a `for-each` loop.

Multi-Dimensional Arrays

Key Points

- Arrays can contain other arrays (2D or more).
- Commonly used for tables, matrices, or grids.
- Each inner array can have a different length (jagged arrays).

Code Snippet

```
int[][] matrix = {  
    {1, 2, 3},  
    {4, 5, 6}  
};  
System.out.println(matrix[1][2]); // prints 6
```

Exercise 05 – TwoDArray.java

Create a 3x3 matrix of integers. Print the diagonal elements.

Jagged (Ragged) Arrays

Key Points

- A jagged array is an array of arrays with varying inner sizes.
- Useful when rows have unequal data lengths.

Code Snippet

```
int[][] jagged = new int[3][];  
jagged[0] = new int[2];  
jagged[1] = new int[4];  
jagged[2] = new int[3];  
System.out.println("Row 1 length: " + jagged[0].length);
```

Exercise 06 – JaggedArray.java

Create a jagged array with 3 rows (2, 3, and 4 elements). Assign values and print them using nested loops.

Array of Objects

Key Points

- Arrays can store references to objects.
- Each element must be initialized individually.

Code Snippet

```
String[] langs = new String[3];  
langs[0] = "Java";  
langs[1] = "Python";  
langs[2] = "Go";  
for (String lang : langs)  
    System.out.println(lang);
```

Exercise 07 – ObjectArray.java

Create an array of `Book` objects (title only). Assign 3 titles and print each title.

Common Array Utilities

Key Points

- `java.util.Arrays` provides helper methods.
- `Arrays.toString(arr)` → readable output.
- `Arrays.sort(arr)` → sorts array.
- `Arrays.equals(a,b)` → compares content.

Code Snippet

```
import java.util.Arrays;
int[] data = {5, 2, 9, 1};
Arrays.sort(data);
System.out.println(Arrays.toString(data)); // [1, 2, 5, 9]
```

Exercise 08 – ArrayUtilities.java

Create an integer array, sort it, and print both before and after sorting using `Arrays.toString()`.

Module – Collection Interfaces (List, Set, Map, Queue, Deque)

Introduction to Collections

Key Points

- The Collections Framework provides classes and interfaces for storing groups of objects.
- Located in the package `java.util`.
- Provides dynamic data structures — arrays are fixed size, collections can grow or shrink.
- Common root interface: `Collection<E>` (except for `Map`, which is separate).

Code Snippet

```
import java.util.*;  
Collection<String> names = new ArrayList<>();  
names.add("Ravi");  
names.add("Meena");  
System.out.println(names);
```

Exercise 01 – CollectionIntro.java

Create a `Collection<String>` and add three language names. Print all elements.

List Interface

Key Points

- List preserves insertion order and allows duplicate elements.
- Elements can be accessed via index.
- Implementations: ArrayList, LinkedList, Vector, Stack.

Code Snippet

```
import java.util.*;  
List<String> cities = new ArrayList<>();  
cities.add("Bangalore");  
cities.add("Mumbai");  
cities.add("Bangalore");  
System.out.println(cities.get(1)); // Mumbai  
System.out.println(cities);        // [Bangalore, Mumbai, Bangalore]
```

Exercise 02 – ListExample.java

Create an `ArrayList<Integer>` with 5 numbers. Remove one element and print the updated list.

Set Interface

Key Points

- Set does not allow duplicates.
- Order may or may not be preserved depending on the implementation.
- Implementations: HashSet, LinkedHashSet, TreeSet.

Code Snippet

```
import java.util.*;  
Set<String> fruits = new HashSet<>();  
fruits.add("Apple");  
fruits.add("Banana");  
fruits.add("Apple");  
System.out.println(fruits); // only one Apple
```

Exercise 03 – SetExample.java

Create a `LinkedHashSet<String>` of 4 color names and print them. Observe that duplicates are ignored.

Map Interface

Key Points

- Map stores key-value pairs (not part of Collection hierarchy).
- Keys must be unique, values can be duplicate.
- Common implementations: HashMap, LinkedHashMap, TreeMap.
- Accessed using keys, not indices.

Code Snippet

```
import java.util.*;  
Map<Integer, String> students = new HashMap<>();  
students.put(101, "Rohit");  
students.put(102, "Anjali");  
students.put(101, "Kiran"); // overwrites previous value  
System.out.println(students);  
System.out.println(students.get(102)); // Anjali
```

Exercise 04 – MapExample.java

Create a Map<String, Integer> of 3 course names and their duration in hours. Print each entry using a for-each loop.

Queue Interface

Key Points

- Queue follows FIFO (First-In, First-Out) order.
- Used for processing tasks, messaging, and buffering.
- Common implementations: LinkedList, PriorityQueue.
- Methods: add(), offer(), poll(), peek(), remove().

Code Snippet

```
import java.util.*;  
Queue<String> queue = new LinkedList<>();  
queue.add("Task1");  
queue.add("Task2");  
queue.add("Task3");  
System.out.println(queue.poll()); // removes Task1  
System.out.println(queue.peek()); // shows Task2
```

Exercise 05 – QueueExample.java

Create a `Queue<String>` of 3 tasks. Process each task until the queue is empty using `poll()`.

Deque Interface

Key Points

- Deque stands for Double-Ended Queue.
- Elements can be added or removed from both ends.
- Can function as queue (FIFO) or stack (LIFO).
- Common implementations: ArrayDeque, LinkedList.

Code Snippet

```
import java.util.*;  
Deque<String> deque = new ArrayDeque<>();  
deque.addFirst("First");  
deque.addLast("Last");  
System.out.println(deque.removeLast()); // removes "Last"  
System.out.println(deque.removeFirst()); // removes "First"
```

Exercise 06 – DequeExample.java

Create an `ArrayDeque<Integer>` and perform push/pop operations to simulate a stack.

Choosing the Right Collection

Key Points

- List → ordered, allows duplicates.
- Set → no duplicates, unordered or sorted.
- Map → key-value pairs.
- Queue → FIFO processing.
- Deque → double-ended, can behave like stack or queue.

Code Snippet

```
List<String> list = new ArrayList<>();  
Set<String> set = new HashSet<>();  
Map<Integer, String> map = new HashMap<>();  
Queue<String> queue = new LinkedList<>();  
Deque<String> deque = new ArrayDeque<>();
```

Exercise 07 – CollectionChooser.java

Given a requirement table, choose appropriate collection types for each use case (unique IDs, ordered playlist, task queue, etc.).

Module – Implementations (ArrayList, LinkedList, HashSet, TreeSet, HashMap, TreeMap)

ArrayList

Key Points

- Ordered, resizable array implementation of List.
- Fast random access (index-based), slower insert/delete in the middle.
- Allows duplicates and null values.
- Backed by a dynamically growing array.

Code Snippet

```
import java.util.*;  
List<String> names = new ArrayList<>();  
names.add("Ravi");  
names.add("Meena");  
names.add("Ravi");  
System.out.println(names);           // [Ravi, Meena, Ravi]  
System.out.println(names.get(1));    // Meena
```

Exercise 01 – ArrayListDemo.java

Create an ArrayList of integers. Add 5 elements, remove one, and print the remaining list.

LinkedList

Key Points

- Doubly-linked list implementation of List and Deque.
- Faster insertion and deletion, slower random access.
- Can function as a queue or stack.
- Allows duplicates and nulls.

Code Snippet

```
import java.util.*;  
LinkedList<String> items = new LinkedList<>();  
items.add("pen");  
items.add("book");  
items.addFirst("bag");  
items.addLast("bottle");  
System.out.println(items); // [bag, pen, book, bottle]  
items.removeFirst();  
System.out.println(items); // [pen, book, bottle]
```

Exercise 02 – LinkedListDemo.java

Use a LinkedList to store 5 course names. Add one at the beginning and one at the end, then print all elements.

HashSet

Key Points

- Implements Set — stores unique elements only.
- Unordered and unsorted (hash-based).
- Allows one null element.
- Uses hash codes for fast lookup and insertion.

Code Snippet

```
import java.util.*;  
Set<String> colors = new HashSet<>();  
colors.add("Red");  
colors.add("Blue");  
colors.add("Red");  
System.out.println(colors); // duplicates ignored
```

Exercise 03 – HashSetDemo.java

Create a HashSet of student names. Try adding duplicates and print the final set.

TreeSet

Key Points

- Sorted, unique collection based on a balanced tree.
- Maintains ascending order by default.
- Does not allow null values.
- Implements NavigableSet for range operations.

Code Snippet

```
import java.util.*;  
Set<Integer> numbers = new TreeSet<>();  
numbers.add(30);  
numbers.add(10);  
numbers.add(20);  
System.out.println(numbers); // [10, 20, 30]
```

Exercise 04 – TreeSetDemo.java

Create a TreeSet of city names. Add elements and display them in ascending order.

HashMap

Key Points

- Implements Map using hashing.
- Stores key-value pairs, keys must be unique.
- Allows one null key and multiple null values.
- Order of entries is not guaranteed.

Code Snippet

```
import java.util.*;  
Map<String, Integer> marks = new HashMap<>();  
marks.put("Math", 90);  
marks.put("Science", 88);  
marks.put("Math", 95); // overwrites existing key  
System.out.println(marks); // {Science=88, Math=95}
```

Exercise 05 – HashMapDemo.java

Create a HashMap of employee IDs and names. Add 3 entries and print them using a for-each loop.

TreeMap

Key Points

- Sorted map based on Red-Black tree structure.
- Keys are stored in natural ascending order by default.
- Does not allow null keys but allows null values.
- Useful when key ordering matters.

Code Snippet

```
import java.util.*;  
Map<Integer, String> students = new TreeMap<>();  
students.put(103, "Neha");  
students.put(101, "Rohan");  
students.put(102, "Priya");  
System.out.println(students); // {101=Rohan, 102=Priya, 103=Neha}
```

Exercise 06 – TreeMapDemo.java

Create a TreeMap of roll numbers and names. Add 4 entries and print them in key order.

Comparison Summary

Key Points

- ArrayList → dynamic, fast access, slower inserts.
- LinkedList → best for frequent inserts/removals.
- HashSet → unique, unordered elements.
- TreeSet → unique and sorted elements.
- HashMap → key-value pairs, unordered.
- TreeMap → key-value pairs, sorted by key.

Code Snippet

```
System.out.println("ArrayList → ordered & allows duplicates");
System.out.println("LinkedList → bidirectional list");
System.out.println("HashSet → unique unordered set");
System.out.println("TreeSet → unique sorted set");
System.out.println("HashMap → unique keys, unordered");
System.out.println("TreeMap → unique keys, sorted");
```

Exercise 07 – CollectionSummary.java

Match each data requirement (unique set, sorted keys, indexed access, etc.) to the appropriate implementation and justify your choice.

Module – Ordering vs Sorting, Comparator, Comparable

Ordering vs Sorting

Key Points

- **Sorting** means arranging elements in a specific sequence (ascending or descending).
- **Ordering** defines *how* elements are compared to decide that sequence.
- Ordering can be **natural** (defined by the class itself) or **custom** (provided externally).
- Sorting in Java uses either **Comparable** or **Comparator** to define ordering logic.

Code Snippet

```
import java.util.*;  
List<Integer> nums = Arrays.asList(5, 2, 8, 1);  
Collections.sort(nums);  
System.out.println(nums); // [1, 2, 5, 8]
```

Exercise 01 – OrderingVsSorting.java

Create a list of integers and sort them in ascending and then in descending order. Print both results.

Comparable Interface (Natural Ordering)

Key Points

- Classes implement **Comparable<T>** to define their natural order.
- Must override the method `compareTo(T o)`.
- Commonly used in built-in classes like String, Integer, Double, etc.
- Returns negative, zero, or positive value depending on comparison result.

Code Snippet

```
class Student implements Comparable<Student> {
    int id;
    String name;
    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
    public int compareTo(Student other) {
        return this.id - other.id; // ascending by id
    }
    public String toString() { return id + " - " + name; }
}

List<Student> list = Arrays.asList(
    new Student(102, "Meena"),
    new Student(101, "Ravi")
);
Collections.sort(list);
System.out.println(list); // [101 - Ravi, 102 - Meena]
```

Exercise 02 – ComparableDemo.java

Create a class Product with fields `price` and `name`. Implement Comparable to sort by price (lowest to highest).

Comparator Interface (Custom Ordering)

Key Points

- **Comparator<T>** defines custom comparison logic outside the class.
- Used when you can't or don't want to modify the class source code.
- Must override `compare(T o1, T o2)` method.
- Can define multiple sorting strategies for the same class.

Code Snippet

```
import java.util.*;

class Student {
    int id; String name;
    Student(int id, String name) { this.id = id; this.name = name; }
    public String toString() { return id + " - " + name; }
}

Comparator<Student> nameSorter = (a, b) -> a.name.compareTo(b.name);
List<Student> list = Arrays.asList(
    new Student(102, "Meena"),
    new Student(101, "Ravi")
);
Collections.sort(list, nameSorter);
System.out.println(list); // [102 - Meena, 101 - Ravi]
```

Exercise 03 – ComparatorDemo.java

Create a `Student` class with fields `name` and `marks`. Use `Comparator` to sort by marks descending and then by name alphabetically.

Comparator vs Comparable

Key Points

- Comparable defines **default (natural)** ordering inside the class.
- Comparator defines **external/custom** ordering outside the class.
- Collections like `TreeSet` and `TreeMap` use either `Comparable` or `Comparator` for ordering.
- Comparator allows multiple sorting strategies; Comparable allows one.

Code Snippet

```
// Comparable - natural ordering
Collections.sort(list);

// Comparator - custom ordering
Collections.sort(list, (a, b) -> a.name.compareTo(b.name));
```

Exercise 04 – CompareBoth.java

Create a `Book` class implementing `Comparable` (sort by title). Then create a `Comparator` to sort by price and print both lists.

Using Comparator with Streams (Java 8+)

Key Points

- Comparator is often used with streams and lambda expressions.
- Built-in static methods: `Comparator.comparing()`, `thenComparing()`, `reversed()`.
- Works seamlessly with `List.sort()` and `Stream.sorted()`.

Code Snippet

```
import java.util.*;  
List<String> names = Arrays.asList("Ravi", "Anjali", "Meena", "Amit");  
names.stream()  
    .sorted(Comparator.comparing(String::length).reversed())  
    .forEach(System.out::println);
```

Exercise 05 – StreamSort.java

Create a list of employee names. Sort them by name length descending using streams and print the result.

Key Rules for Certification

Key Points

- `compareTo()` must return negative, zero, or positive (no boolean).
- If two objects are equal per `compareTo`, they should be equal per `equals()`.
- `Comparator` can be passed directly to `Collections.sort()` or `List.sort()`.
- `TreeSet` and `TreeMap` use either `Comparable` or `Comparator` for ordering.
- Using both can lead to inconsistent ordering if logic differs.

Code Snippet

```
Set<Integer> set = new TreeSet<>(Comparator.reverseOrder());
set.addAll(Arrays.asList(10, 5, 20));
System.out.println(set); // [20, 10, 5]
```

Exercise 06 – TreeSetOrdering.java

Create a `TreeSet` of integers in descending order using a custom `Comparator` and print all elements.

Module – Immutable Collections (List.of, Set.of, Map.of)

Introduction to Immutable Collections

Key Points

- Immutable collections are **read-only**; once created, they **cannot be modified**.
- Introduced in **Java 9** via `List.of()`, `Set.of()`, and `Map.of()`.
- They offer better **safety**, **thread-safety**, and **memory efficiency** than unmodifiable wrappers.
- Any attempt to add, remove, or update elements throws `UnsupportedOperationException`.

Code Snippet

```
import java.util.*;  
List<String> fruits = List.of("Apple", "Banana", "Mango");  
System.out.println(fruits);  
// fruits.add("Orange"); // throws UnsupportedOperationException
```

Exercise 01 – ImmutableIntro.java

Create a List of 3 car brands using `List.of()` and try to modify it. Observe the exception message.

List.of()

Key Points

- Creates an **immutable list** with specified elements.
- Cannot contain null elements — throws `NullPointerException`.
- Order is preserved, duplicates allowed (since `List` permits duplicates).

Code Snippet

```
List<Integer> numbers = List.of(10, 20, 30);  
System.out.println(numbers.get(1)); // 20
```

Exercise 02 – ListOfDemo.java

Create a list of integers using `List.of()` and print each value using a for-each loop.

Set.of()

Key Points

- Creates an **immutable set**.
- Duplicate elements are **not allowed** — duplicates cause `IllegalArgumentException`.
- Order of elements is **undefined**.
- Null elements are not permitted.

Code Snippet

```
Set<String> cities = Set.of("Delhi", "Mumbai", "Pune");  
// Set<String> invalid = Set.of("A", "B", "A"); // throws  
IllegalArgumentException  
System.out.println(cities);
```

Exercise 03 – SetOfDemo.java

Create a `Set` of five programming languages using `Set.of()`. Try adding a duplicate to see what happens.

Map.of()

Key Points

- Creates an **immutable map** with up to 10 key-value pairs.
- For more than 10 entries, use `Map.ofEntries()`.
- Duplicate keys cause `IllegalArgumentException`.
- Null keys or values are not allowed.

Code Snippet

```
Map<Integer, String> courses = Map.of(  
    101, "Java",  
    102, "Python",  
    103, "Go"  
);  
System.out.println(courses.get(102)); // Python
```

Exercise 04 – MapOfDemo.java

Create a Map of subject codes and names using `Map.of()`. Attempt to insert a null key and note the behavior.

Map.ofEntries()

Key Points

- Used when creating maps with more than 10 entries.
- Accepts `Map.entry(key, value)` pairs.
- Also returns an immutable map.

Code Snippet

```
import static java.util.Map.entry;
Map<Integer, String> largeMap = Map.ofEntries(
    entry(1, "One"), entry(2, "Two"), entry(3, "Three"), entry(4, "Four")
);
System.out.println(largeMap);
```

Exercise 05 – MapOfEntriesDemo.java

Create a `Map<Integer, String>` with more than 10 entries using `Map.ofEntries()` and print all values.

Comparing with Unmodifiable Wrappers

Key Points

- `Collections.unmodifiableList()` returns a **view** of an existing list; modifying the original affects it.
- `List.of()` creates a **truly immutable** collection — independent and fixed.
- Immutable collections are ideal for constants, configuration data, and safe APIs.

Code Snippet

```
List<String> modifiable = new ArrayList<>();
modifiable.add("X");
List<String> unmodifiable = Collections.unmodifiableList(modifiable);
modifiable.add("Y");
System.out.println(unmodifiable); // reflects change → [X, Y]

List<String> trulyImmutable = List.of("A", "B");
System.out.println(trulyImmutable); // [A, B]
// trulyImmutable.add("C"); // throws exception
```

Exercise 06 – ImmutableComparison.java

Create one unmodifiable list and one immutable list. Modify the original list and observe which one changes.

Certification Tips

Key Points

- `List.of()`, `Set.of()`, and `Map.of()` **do not allow nulls** or duplicates.
- Attempting structural modification throws `UnsupportedOperationException`.
- Immutable collections are **thread-safe for read-only use**.
- `Map.ofEntries()` supports any number of entries.
- Common exam trap: confusing `Collections.unmodifiableXxx()` (view) with `Xxx.of()` (immutable copy).

Code Snippet

```
Set<String> set = Set.of("A", "B", "C");  
// set.remove("A"); // UnsupportedOperationException  
System.out.println(set.contains("B")); // true
```

Exercise 07 – ImmutablePractice.java

Summarize the differences between mutable, unmodifiable, and immutable collections using examples of `List`, `Set`, and `Map`.

Module – Fail-Fast Iterators and ConcurrentModificationException

Introduction to Fail-Fast Behavior

Key Points

- Most iterators in the **Collections Framework** are *fail-fast*.
- Fail-fast iterators detect **structural modifications** (add, remove, clear) made to a collection after the iterator is created.
- If modification occurs outside the iterator's own methods, the iterator throws **ConcurrentModificationException (CME)**.
- Designed to prevent **unpredictable behavior** during concurrent modifications.

Code Snippet

```
import java.util.*;
List<String> names = new ArrayList<>();
names.add("A");
names.add("B");
names.add("C");

for (String n : names) {
    if (n.equals("B"))
        names.remove(n); // triggers ConcurrentModificationException
}
```

Exercise 01 – FailFastDemo.java

Create a list of 5 strings and try removing one element inside an enhanced for-loop. Observe the exception message.

Understanding ConcurrentModificationException

Key Points

- **ConcurrentModificationException** is a **runtime exception** thrown when a collection is modified structurally during iteration.
- Structural modification means adding, removing, or resizing the collection.
- It is detected using an internal counter called **modCount** in collection classes.
- If the iterator detects a change in modCount, it immediately throws the exception.

Code Snippet

```
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4));
Iterator<Integer> it = nums.iterator();
nums.add(5); // modifies collection after iterator creation
while (it.hasNext())
    System.out.println(it.next()); // throws
ConcurrentModificationException
```

Exercise 02 – ConcurrentModDemo.java

Write code to create a list and iterate using an explicit `Iterator`. Modify the list during iteration and see when CME occurs.

Safe Removal During Iteration

Key Points

- To safely modify a collection during iteration, use the iterator's own **remove()** method.
- `iterator.remove()` removes the last returned element safely without changing `modCount` unexpectedly.
- Always avoid structural modification outside the iterator while iterating.

Code Snippet

```
List<String> list = new ArrayList<>(List.of("A", "B", "C"));
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    String val = it.next();
    if (val.equals("B"))
        it.remove(); // safe removal
}
System.out.println(list); // [A, C]
```

Exercise 03 – SafeRemoval.java

Write code that removes elements safely using an iterator. Print the final collection after removal.

Fail-Fast vs Fail-Safe Iterators

Key Points

- **Fail-Fast:** throws `ConcurrentModificationException` when collection is structurally modified.
- **Fail-Safe:** operates on a **clone or snapshot** of the collection, so no exception occurs.
- Fail-safe iterators belong to **concurrent collections** in `java.util.concurrent`.
- Fail-safe iterators do not reflect changes made after iteration begins.

Code Snippet

```
import java.util.concurrent.*;
CopyOnWriteArrayList<String> list = new CopyOnWriteArrayList<>();
list.add("A");
list.add("B");
for (String s : list) {
    list.add("C"); // no ConcurrentModificationException
}
System.out.println(list); // [A, B, C, C]
```

Exercise 04 – FailSafeDemo.java

Use a `CopyOnWriteArrayList` and modify it during iteration. Compare behavior with `ArrayList`.

Common Fail-Fast Collections

Key Points

- All standard **non-concurrent collections** like `ArrayList`, `HashSet`, `LinkedList`, `HashMap`, and `TreeMap` are **fail-fast**.
- Fail-fast iterators are meant for **single-threaded safety**, not concurrency.
- If multiple threads modify a collection, synchronization or concurrent collections must be used.

Code Snippet

```
Map<Integer, String> map = new HashMap<>();
map.put(1, "A");
map.put(2, "B");
for (Integer key : map.keySet()) {
    map.put(3, "C"); // throws ConcurrentModificationException
}
```

Exercise 05 – MapFailFast.java

Create a `HashMap` and attempt to modify it while iterating through its keys. Capture and print the exception.

Certification Tips

Key Points

- Fail-fast behavior is a **runtime detection mechanism**, not guaranteed in all cases.
- Structural modifications cause CME; non-structural changes (like updating an existing value in a Map) do not.
- Fail-safe iterators (CopyOnWriteArrayList, ConcurrentHashMap) work on a separate snapshot.
- Always prefer concurrent collections for **multi-threaded** code.

Code Snippet

```
import java.util.concurrent.*;
Map<Integer, String> cmap = new ConcurrentHashMap<>();
cmap.put(1, "One");
cmap.put(2, "Two");
for (Integer key : cmap.keySet()) {
    cmap.put(3, "Three"); // allowed, no exception
}
System.out.println(cmap); // {1=One, 2=Two, 3=Three}
```

Exercise 06 – ConcurrentCollections.java

Replace a regular collection with its concurrent equivalent and verify that the iteration completes without throwing CME.

Module – Wrapper Collections (`synchronizedList`, `unmodifiableList`)

Introduction to Wrapper Collections

Key Points

- Wrapper collections are created using static methods of the `Collections` class.
- They wrap an existing collection to add **thread-safety** or **read-only (unmodifiable)** behavior.
- Original data is still stored inside the wrapped collection.
- Examples include `Collections.synchronizedList()`, `synchronizedSet()`, `unmodifiableList()`, and `unmodifiableMap()`.

Code Snippet

```
import java.util.*;  
List<String> data = new ArrayList<>();  
List<String> syncList = Collections.synchronizedList(data);  
List<String> readOnly = Collections.unmodifiableList(data);
```

Exercise 01 – WrapperIntro.java

Create an `ArrayList`, wrap it with `synchronizedList` and `unmodifiableList`, and print both.

Collections.synchronizedList()

Key Points

- Returns a **thread-safe wrapper** around a given List.
- All operations are synchronized on the internal lock of the list.
- Must use **synchronized blocks** when iterating to avoid ConcurrentModificationException.
- Suitable for low-contention multi-threaded environments.

Code Snippet

```
import java.util.*;  
List<String> list = new ArrayList<>();  
List<String> syncList = Collections.synchronizedList(list);  
  
syncList.add("A");  
syncList.add("B");  
  
// synchronized block required during iteration  
synchronized (syncList) {  
    for (String s : syncList) {  
        System.out.println(s);  
    }  
}
```

Exercise 02 – SyncListDemo.java

Create a synchronized list, add 3 elements, and iterate safely inside a synchronized block.

Collections.unmodifiableList()

Key Points

- Creates a **read-only view** of the original list.
- Any attempt to modify the unmodifiable list (add, remove, set) throws `UnsupportedOperationException`.
- If the underlying list changes, the unmodifiable view reflects those changes.
- Often used for returning safe, immutable views of internal data.

Code Snippet

```
import java.util.*;  
List<String> list = new ArrayList<>();  
list.add("X");  
List<String> readOnly = Collections.unmodifiableList(list);  
  
System.out.println(readOnly); // [X]  
// readOnly.add("Y"); // UnsupportedOperationException  
  
list.add("Z"); // modifies original list  
System.out.println(readOnly); // [X, Z]
```

Exercise 03 – UnmodifiableListDemo.java

Create a modifiable list, wrap it with `unmodifiableList`, and test what happens when you modify the original and the wrapper.

Other Wrapper Methods

Key Points

- `Collections.synchronizedSet()`, `Collections.synchronizedMap()` → thread-safe versions of sets and maps.
- `Collections.unmodifiableSet()`, `Collections.unmodifiableMap()` → read-only views.
- Wrapper collections are **not truly immutable** — they only restrict external modification.
- For truly immutable data, use `List.of()`, `Set.of()`, or `Map.of()`.

Code Snippet

```
Map<Integer, String> map = new HashMap<>();
map.put(1, "One");
Map<Integer, String> readOnlyMap = Collections.unmodifiableMap(map);
map.put(2, "Two"); // allowed, affects view
// readOnlyMap.put(3, "Three"); // throws UnsupportedOperationException
System.out.println(readOnlyMap); // {1=One, 2=Two}
```

Exercise 04 – WrapperMapDemo.java

Create a `HashMap`, wrap it in `unmodifiableMap`, then modify the original map and observe if changes are visible through the wrapper.

Comparing Wrapper vs Immutable Collections

Key Points

- `Collections.unmodifiableList()` creates a **view** — changes in the base list are reflected.
- `List.of()` creates a **truly immutable list** — separate, cannot change even if the original does.
- Synchronized wrappers are for **thread safety**, not immutability.
- For concurrent applications, prefer concurrent collections (e.g., `CopyOnWriteArrayList`).

Code Snippet

```
List<String> base = new ArrayList<>();
base.add("Java");
List<String> unmodifiable = Collections.unmodifiableList(base);
List<String> immutable = List.of("Java");

base.add("Python");
System.out.println(unmodifiable); // [Java, Python]
System.out.println(immutable);    // [Java]
```

Exercise 05 – WrapperVsImmutable.java

Create an `ArrayList` and wrap it using both `unmodifiableList` and `List.of()`. Modify the base list and compare the results of both views.

Certification Tips

Key Points

- Wrapper collections use **decorator pattern** — they wrap and extend behavior.
- `Collections.synchronizedXxx()` → adds synchronization for thread safety.
- `Collections.unmodifiableXxx()` → adds read-only restriction.
- Both are shallow wrappers; modifications through the base collection still affect them.
- Attempting structural modification through the wrapper throws `UnsupportedOperationException`.

Code Snippet

```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3));  
List<Integer> unmod = Collections.unmodifiableList(list);  
list.add(4);  
System.out.println(unmod); // [1, 2, 3, 4]
```

Exercise 06 – CertificationCheck.java

Summarize the differences between synchronized and unmodifiable wrappers, and identify which decorator pattern each uses.

Module – Generic Type Parameters and Wildcards (?, extends, super)

Why Generics

Key Points

- Generics add **type safety** and eliminate **casting**.
- Introduced in Java 5, generics allow classes and methods to operate on **typed objects**.
- Prevent runtime `ClassCastException` by enforcing compile-time checks.
- Improve code reuse and readability.

Code Snippet

```
List<String> names = new ArrayList<>();  
names.add("Ravi");  
// names.add(10); // compile-time error  
String first = names.get(0); // no casting required
```

Exercise 01 – GenericsIntro.java

Create a generic `List<Integer>` and add values. Try to add a `String` and observe the compiler error.

Defining Generic Classes

Key Points

- Use **type parameters** (<T>, <E>, <K, V>) when defining classes or methods.
- T is a placeholder for the actual type supplied during instantiation.
- Generic types can be bounded to restrict the acceptable classes.

Code Snippet

```
class Box<T> {  
    private T item;  
    public void set(T item) { this.item = item; }  
    public T get() { return item; }  
}  
Box<String> box = new Box<>();  
box.set("Hello");  
System.out.println(box.get());
```

Exercise 02 – GenericBox.java

Define a generic class `Container<T>` with `add()` and `get()` methods. Instantiate it for Integer and String types.

Generic Methods

Key Points

- Methods can declare their own generic parameters independent of the class.
- **Syntax:** <T> ReturnType methodName(T param).
- Enables type-safe operations inside utility or static methods.

Code Snippet

```
class Util {  
    public static <T> void printArray(T[] arr) {  
        for (T t : arr) System.out.print(t + " ");  
    }  
}  
String[] langs = {"Java", "Python"};  
Util.printArray(langs);
```

Exercise 03 – GenericMethod.java

Create a generic method that prints any array type. Test it with Integer[] and String[].

Wildcards (?)

Key Points

- A **wildcard** (?) represents an unknown type.
- Useful when you need to read from generic types but don't care about their exact type argument.
- Enables flexibility when working with related generic objects.

Code Snippet

```
List<?> list = List.of("A", "B", "C");  
for (Object o : list) System.out.println(o);  
// list.add("D"); // not allowed
```

Exercise 04 – WildcardDemo.java

Create a `List<?>` and print all its elements. Try to add a new element and note the compiler message.

Upper-Bounded Wildcards (? extends Type)

Key Points

- `? extends T` means the list contains elements of type `T` or its subclasses.
- Read-only access: can safely read `T`, but cannot add new elements.
- Commonly used when a method **reads** generic data.

Code Snippet

```
List<? extends Number> nums = List.of(1, 2.5, 3L);  
for (Number n : nums) System.out.println(n);  
// nums.add(10); // compile-time error
```

Exercise 05 – UpperBound.java

Create a method `printNumbers(List<? extends Number> list)` and call it with `List<Integer>` and `List<Double>`.

Lower-Bounded Wildcards (? super Type)

Key Points

- `? super T` means the list can contain elements of type T or its superclasses.
- Allows **writing** objects of type T into the list.
- Commonly used in methods that **consume** generic data.

Code Snippet

```
List<? super Integer> nums = new ArrayList<Number>();  
nums.add(10);  
nums.add(20);  
// Integer x = nums.get(0); // invalid: type is Object
```

Exercise 06 – LowerBound.java

Create a `List<? super Integer>` and add integers to it. Observe the return type when retrieving elements.

PECS Rule (*Producer Extends, Consumer Super*)

Key Points

- **PECS** = *Producer Extends, Consumer Super*.
- Use `extends` when the collection **produces** data for reading.
- Use `super` when the collection **consumes** data for writing.
- Helps decide correct wildcard direction for method parameters.

Code Snippet

```
void copy(List<? extends Number> src, List<? super Number> dest) {  
    for (Number n : src) dest.add(n);  
}
```

Exercise 07 – PECSRule.java

Write a `copy()` method using `? extends` and `? super`. Pass a `List<Integer>` as source and a `List<Object>` as destination.

Certification Tips

Key Points

- Wildcards (?) cannot be used when declaring generic classes or methods — only for references.
- `List<Object>` and `List<?>` are not the same; `List<?>` is more flexible for unknown types.
- Bounded wildcards enforce read-or-write rules but never both simultaneously.
- Erasure removes type information at runtime; generics are compile-time only.

Code Snippet

```
List<String> list = new ArrayList<>();  
System.out.println(list.getClass() == new ArrayList<Integer>().getClass());  
// true
```

Exercise 08 – TypeErasure.java

Demonstrate type erasure by comparing `ArrayList<String>` and `ArrayList<Integer>` classes and printing the result.

Module – Type Erasure and Compiler Inference Rules

Understanding Type Erasure

Key Points

- Generics in Java exist **only at compile time** — this is called *type erasure*.
- At runtime, all type parameters are **removed (erased)** and replaced with their bounds or `Object`.
- This maintains **backward compatibility** with older (non-generic) Java code.
- The compiled bytecode works on raw types like `List` or `Object`.

Code Snippet

```
List<String> list1 = new ArrayList<>();  
List<Integer> list2 = new ArrayList<>();  
System.out.println(list1.getClass() == list2.getClass()); // true
```

Exercise 01 – ErasureCheck.java

Create a `List<String>` and `List<Double>`. Compare their `getClass()` results and confirm they are identical at runtime.

Erasure Process Internals

Key Points

- During compilation, the compiler replaces each **type parameter** with its **bound**:
 - Unbounded `<T>` → replaced with `Object`.
 - Bounded `<T extends Number>` → replaced with `Number`.
- The compiler inserts **type casts** automatically wherever required.
- This explains why generics cannot use primitive types (e.g., `List<int>`).

Code Snippet

```
class Box<T> {  
    T value;  
    T get() { return value; }  
}  
class AfterErasure {  
    Object value;  
    Object get() { return value; }  
}
```

Exercise 02 – ErasureView.java

Write a generic class with one method and manually rewrite it as the compiler would after type erasure.

Implications of Type Erasure

Key Points

- Type information is not available at runtime — hence **no generic array creation**.
- You cannot use `instanceof` or reflection to check parameterized types.
- Example invalid operations:
 - `new T[]` **or** `new List<String>[5]`
 - `if (obj instanceof List<String>)`
- Use bounded wildcards or `Class<T>` tokens instead when you need type awareness.

Code Snippet

```
if (obj instanceof List<?>) { // legal
    System.out.println("It's a List of something!");
}
```

Exercise 03 – GenericLimitations.java

Attempt to create a generic array and handle the compiler error. Replace it with a `List` alternative.

Bridge Methods (Generated by Compiler)

Key Points

- When erasure causes method signatures to clash, the compiler adds **bridge methods** to preserve polymorphism.
- Bridge methods ensure overridden generic methods remain type-compatible after erasure.
- Usually visible only in decompiled bytecode, not in source code.

Code Snippet

```
class Parent<T> { T get() { return null; } }  
class Child extends Parent<String> {  
    @Override String get() { return "Java"; } // compiler creates bridge  
    method  
}
```

Exercise 04 – BridgeDemo.java

Compile and decompile a subclass overriding a generic method. Identify the bridge method generated by the compiler.

Type Inference Overview

Key Points

- Type inference allows the compiler to **deduce generic types automatically**.
- Introduced in Java 7 with the **diamond operator (<>)** and enhanced in Java 8 with **method inference**.
- Works for variable declarations, method calls, and constructors.
- Reduces verbosity while maintaining type safety.

Code Snippet

```
Map<Integer, String> map = new HashMap<>(); // type inferred
```

Exercise 05 – InferenceIntro.java

Use the diamond operator with `ArrayList`, `HashMap`, and `TreeSet` to create type-safe collections without repeating generic types.

Method Type Inference

Key Points

- The compiler infers type arguments from **method parameters or return type context**.
- Generic methods often rely on the type of passed arguments to infer `<T>`.
- Explicit type arguments can still be provided when inference fails.

Code Snippet

```
class Util {  
    static <T> T pickFirst(T a, T b) { return a; }  
}  
var result = Util.pickFirst("Hello", "World"); // T inferred as String
```

Exercise 06 – InferenceMethod.java

Write a generic method `max(T a, T b, Comparator<T>)` and call it without specifying `<T>` explicitly.

Target Type Inference (Java 8+)

Key Points

- The **target type** (expected type of an expression) guides inference.
- Common in lambda expressions and method references.
- Allows writing concise functional code with generics.

Code Snippet

```
List<String> list = List.of("A", "B", "C");  
list.forEach(System.out::println); // target type inferred as  
Consumer<String>
```

Exercise 07 – TargetTypeInference.java

Create a lambda expression that sorts a list of strings using `List.sort()` without specifying `Comparator` type.

Certification Tips

Key Points

- Generics use **compile-time checking** only; type erasure removes all type info at runtime.
- Bridge methods may appear in decompiled code — remember they are compiler-generated.
- Inference rules rely on context: argument types, assignment target, and return type.
- You cannot overload methods based only on generic type parameters due to erasure.
- Always prefer explicit bounds (`<T extends Number>`) for clarity in exams.

Code Snippet

```
class Test {  
    // Compile error: both methods erase to same signature  
    void print(List<String> x) {}  
    // void print(List<Integer> x) {} // invalid after erasure  
}
```

Exercise 08 – ErasureConflict.java

Try overloading two methods differing only in generic parameter types and observe the compile-time error.

Module – Generic Methods and Bounded Type Parameters

What Are Generic Methods

Key Points

- A **generic method** defines its own type parameter(s) independent of the class it belongs to.
- Declared with a type parameter section before the return type: `<T> ReturnType methodName(...)`.
- Enables reusable logic for any data type while preserving type safety.
- Works in both static and instance methods.

Code Snippet

```
class Util {  
    public static <T> void printItem(T item) {  
        System.out.println(item);  
    }  
}  
Util.printItem("Java");  
Util.printItem(123);
```

Exercise 01 – GenericMethodIntro.java

Write a generic method `display(T value)` that prints any type of argument and test it with a String and an Integer.

Multiple Type Parameters

Key Points

- You can declare multiple type parameters: `<T, U, V>`.
- Useful when working with pairs, mappings, or combined operations.
- Each parameter can have its own bounds or remain unbounded.

Code Snippet

```
class PairUtil {  
    public static <K, V> void showPair(K key, V value) {  
        System.out.println(key + " = " + value);  
    }  
}  
  
PairUtil.showPair("Language", "Java");  
PairUtil.showPair(1, 99.9);
```

Exercise 02 – MultiTypeParams.java

Write a generic method with `<X, Y>` that takes two parameters of different types and prints both.

Bounded Type Parameters

Key Points

- Bounded type parameters restrict the acceptable types using **extends**.
- `<T extends Number>` → T can be Number or any subclass (Integer, Double, etc.).
- Allows access to methods and properties of the bounded type.
- Multiple bounds can be combined using `&` (interfaces first if mixed).

Code Snippet

```
class MathUtil {  
    public static <T extends Number> double square(T value) {  
        return value.doubleValue() * value.doubleValue();  
    }  
}  
System.out.println(MathUtil.square(5));    // 25.0  
System.out.println(MathUtil.square(3.2));  // 10.24
```

Exercise 03 – BoundedMethod.java

Create a method `<T extends Number>` that returns the sum of two numbers using `doubleValue()`.

Multiple Bounds

Key Points

- A type parameter can extend **one class** and implement **multiple interfaces**.
- **Syntax:** <T extends Class & Interface1 & Interface2>.
- The class must appear first, followed by interfaces.

Code Snippet

```
interface Printable { void print(); }
class Document implements Printable { public void print() {
System.out.println("Doc"); } }

class GenericUtil {
    public static <T extends Printable> void show(T obj) {
        obj.print();
    }
}
GenericUtil.show(new Document());
```

Exercise 04 – MultiBound.java

Define a class that implements two interfaces. Create a generic method with multiple bounds and call it using that class.

Generic Return Types

Key Points

- Generic methods can return the same type they accept.
- Helps ensure type consistency between input and output.
- Useful for builders, factories, and fluent APIs.

Code Snippet

```
class Factory {  
    public static <T> T getInstance(T value) {  
        return value;  
    }  
}  
  
String result = Factory.getInstance("Hello");  
Integer num = Factory.getInstance(10);  
System.out.println(result + " " + num);
```

Exercise 05 – GenericReturn.java

Create a method `<T> T echo(T input)` that returns the same object passed in. Test it with `String` and `Double`.

Generic Methods vs Generic Classes

Key Points

- A **generic class** defines type parameters at class level and applies them to all methods.
- A **generic method** defines its own parameters independent of the class.
- You can have generic methods inside non-generic classes and vice versa.
- Generic methods are ideal for utility classes like `Collections` or `Arrays`.

Code Snippet

```
class Printer {  
    public <T> void print(T data) {  
        System.out.println(data);  
    }  
}  
  
Printer p = new Printer();  
p.print("Learn Java");  
p.print(42);
```

Exercise 06 – GenericComparison.java

Create a non-generic class with a generic method that prints the runtime type of any argument.

Wildcards vs Bounded Type Parameters

Key Points

- `? extends T` and `<T extends X>` look similar but serve different purposes.
- Bounded types are used **when defining methods**; wildcards are used **when calling them**.
- Example: `<T extends Number>` **defines**; `List<? extends Number>` **applies**.
- Certification tip: Bounded parameters allow you to access class methods; wildcards only restrict access.

Code Snippet

```
class Stats {  
    public static <T extends Number> double average(List<T> list) {  
        double sum = 0;  
        for (T t : list) sum += t.doubleValue();  
        return sum / list.size();  
    }  
}
```

Exercise 07 – StatsAverage.java

Create a generic method `<T extends Number>` that calculates the average of a list of numeric values.

Certification Tips

Key Points

- Always declare type parameters **before** the return type.
- `extends` in a type parameter means “is-a” bound (not just inheritance).
- Multiple bounds must list the **class first**, followed by interfaces.
- Generic methods can be invoked with **explicit** type arguments if inference fails.
- You cannot use primitives as generic type arguments — only reference types.

Code Snippet

```
class Demo {  
    public static <T extends Comparable<T>> T min(T a, T b) {  
        return (a.compareTo(b) < 0) ? a : b;  
    }  
}  
System.out.println(Demo.min("Java", "Python")); // Java
```

Exercise 08 – MinFinder.java

Write a method `<T extends Comparable<T>>` that returns the smaller of two elements. Test with Strings and Integers.

Module – Common Generic Pitfalls and Raw Type Usage

Introduction to Generic Pitfalls

Key Points

- Generics provide type safety but are limited by **type erasure** and **backward compatibility**.
- Misuse often leads to warnings or unexpected runtime behavior.
- Common issues include raw types, type casting, mixing generic and non-generic code, and unchecked operations.

Code Snippet

```
List list = new ArrayList(); // raw type
list.add("Java");
list.add(10); // allowed, but unsafe
String s = (String) list.get(1); // ClassCastException at runtime
```

Exercise 01 – RawTypeError.java

Create a raw list, mix different types, and observe what happens when you cast elements incorrectly.

Raw Types

Key Points

- A **raw type** is a generic type used **without type parameters** (e.g., `List` instead of `List<String>`).
- Exists for backward compatibility with pre-Java 5 code.
- Disables compile-time checks and can cause `ClassCastException`.
- Should always be replaced with parameterized types in modern code.

Code Snippet

```
List raw = new ArrayList();    // raw type
List<String> safe = new ArrayList<>(); // generic

raw.add(100);
safe.add("Hello");
// raw and safe cannot be safely mixed
```

Exercise 02 – RawVsGeneric.java

Create two lists — one raw and one generic — and try assigning one to another. Note compiler warnings.

Mixing Raw and Parameterized Types

Key Points

- Mixing raw and generic types leads to **unchecked warnings**.
- The compiler cannot guarantee type safety when generics are lost.
- Always parameterize both sides of assignments and method calls.

Code Snippet

```
List<String> list = new ArrayList<>();  
list.add("A");  
List raw = list;           // warning: unchecked conversion  
raw.add(100);              // compiles but breaks type safety  
System.out.println(list.get(1)); // ClassCastException
```

Exercise 03 – MixedTypeDemo.java

Assign a generic collection to a raw reference, insert invalid data, and run the program.

Unchecked Casts and Warnings

Key Points

- Casting between generic types often triggers **unchecked warnings**.
- The compiler cannot confirm type compatibility after type erasure.
- Avoid unchecked casts unless absolutely necessary (e.g., legacy interop).

Code Snippet

```
Object obj = new ArrayList<String>();  
List<Integer> nums = (List<Integer>) obj; // unchecked cast warning  
nums.add(10); // may cause runtime exception later
```

Exercise 04 – UncheckedCast.java

Perform an unchecked cast from `Object` to `List<Integer>` and explore the warning message.

Generic Arrays

Key Points

- Arrays are **covariant** and reified (know their type at runtime).
- Generics are **invariant** and erased (type info gone at runtime).
- Hence, arrays of parameterized types cannot be created directly.
- Always prefer using `List<T>` over `T[]` for generic collections.

Code Snippet

```
// List<String>[] arr = new List<String>[5]; // compile-time error
List<?>[] arr = new List<?>[5]; // allowed but unsafe
arr[0] = List.of(1, 2, 3);      // warning: heap pollution
```

Exercise 05 – GenericArray.java

Try creating an array of generic lists. Observe compile-time errors or warnings and explain why they occur.

Heap Pollution

Key Points

- Occurs when a variable of a parameterized type refers to an object of a different type.
- Usually caused by unchecked casts or mixing raw types.
- May lead to runtime `ClassCastException` even if compilation succeeds.

Code Snippet

```
List<String> strings = new ArrayList<>();
List raw = strings;
raw.add(100); // heap pollution
for (String s : strings) { // runtime exception
    System.out.println(s);
}
```

Exercise 06 – HeapPollution.java

Demonstrate heap pollution by adding incompatible data to a list through a raw reference.

Overloading and Type Erasure Conflicts

Key Points

- Methods differing only in type parameters **cannot be overloaded** because of type erasure.
- After compilation, both signatures erase to the same method.
- The compiler will report a duplicate method definition error.

Code Snippet

```
class Demo {  
    // void print(List<String> list) {}  
    // void print(List<Integer> list) {} // compile-time error  
}
```

Exercise 07 – OverloadError.java

Try overloading two methods with the same erased generic signature and view the compiler message.

Generic Type Inference Pitfalls

Key Points

- The compiler may infer `Object` when it cannot determine the correct type.
- Using explicit type arguments can prevent ambiguity.
- Be cautious when chaining generic method calls with mixed return types.

Code Snippet

```
class Util {  
    static <T> T pick(T a, T b) { return a; }  
}  
var x = Util.pick(10, "Hi"); // inferred as Object  
System.out.println(x.getClass()); // class java.lang.Integer (autoboxed)
```

Exercise 08 – InferenceTrap.java

Create a generic method returning one of two arguments of different types. Inspect the inferred type using `getClass()`.

Certification Tips

Key Points

- Avoid using **raw types** in modern Java code — always use parameterized types.
- Do not create arrays of parameterized types.
- Be aware of **unchecked warnings** — they signal potential runtime risks.
- `List<?>` is safer than raw `List` when you need a flexible, read-only reference.
- Generic type information is erased at runtime — always handle with compile-time checks.

Code Snippet

```
List<?> safe = List.of("Java", 100, true);  
System.out.println(safe.get(1)); // valid read-only access  
// safe.add("New"); // compile-time error
```

Exercise 09 – CertificationPractice.java

Create examples showing one compile-time generic warning, one runtime exception, and one safe generic usage. Explain each.